

Programmers often forget about documenting their code. But by taking a little time to add comments, docstrings, and type hints to your code, you can avoid wasting time later. Well-documented code is also easier to maintain.

It's tempting to adopt the opinion that comments and documentation don't matter or are even a disadvantage when writing software. (Conveniently, this view allows programmers to avoid the work of writing documentation.) Don't be fooled; well-written documentation consistently saves you far more time and effort than it takes to write it. It's more common for programmers to stare at a screen of inscrutable, uncommented code than to have too much helpful information.

Good comments offer concise, useful, and accurate information to programmers who need to read the code at a later time and understand what it does. These comments should explain the original programmer's intent and summarize small sections of code rather than state the obvious about what a single line of code does. Comments sometimes offer detailed accounts of lessons the programmer learned while writing the code. This valuable information might spare future maintainers from having to relearn these lessons the hard way.

Docstrings, a Python-specific kind of comment, are multiline strings that appear immediately after the `class` or `def` statement, or at the top of the module. Documentation tools, such as Python's built-in `help()` function, can extract docstrings to provide specific information about the purpose of the class, function, or module.

Introduced in Python 3.5, type hints bring gradual typing to Python code. Gradual typing allows the programmer to apply the bug-detection benefits of static typing while maintaining the flexibility of dynamic typing. The Python interpreter ignores type hints, because Python doesn't have runtime type checking. Even so, static type-checking tools use type hints to analyze the source code when it isn't running. Type checkers, such as Mypy, can ensure that you aren't assigning invalid values to the variables you pass to functions. This saves you time and effort by preventing a broad category of bugs.